

3 차 프로젝트 · 인프라 설계 보고서

[서비스명]

Kubernetes + AWS — 컨테이너 오케스트레이션 인프라 설계 · 구축 · 운영

팀 [팀명] · 2026. 07 · 초안 v0.1

※ [대괄호] 항목은 팀 논의 후 확정



AWS



EKS



Terraform



Docker



Actions

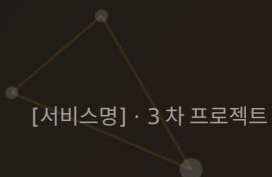
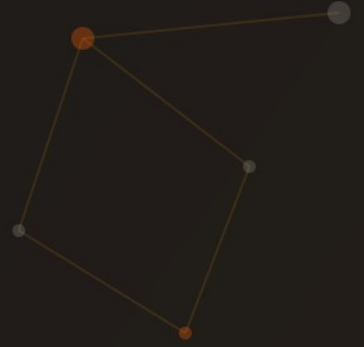


Grafana



목차

※ Word 에서 열람 시 목차 필드를 업데이트하세요 (F9)



1. 프로젝트 개요

3 차 팀 프로젝트로, Kubernetes 와 AWS 를 결합한 컨테이너 오케스트레이션 인프라 위에 웹 서비스를 구축·운영한다. 단순히 "동작하는 인프라"를 만드는 것을 넘어, 오토스케일링·무중단 배포·장애 복구가 실제로 동작함을 부하 테스트와 장애 주입으로 실측 증명하는 것까지를 프로젝트의 범위로 정의한다. 모든 인프라는 Terraform 으로 코드화하며(콘솔 수작업 0 건), 컴퓨트 계층은 처음부터 EKS(Kubernetes) 네이티브로 설계한다.

서비스 개요: [서비스 한 줄 설명 — 예: OO 를 위한 웹 애플리케이션. 프론트 SPA + 백엔드 API + RDB/캐시 구성]

2. 목표

- IaC 100% — 콘솔 수작업 0 건, terraform apply 한 번으로 전체 환경 재현
- EKS 기반 컨테이너 오케스트레이션 — HPA(Pod)와 Cluster Autoscaler(노드)의 2 단 오토스케일링
- GitOps 형 CI/CD — GitHub Actions(OIDC, 액세스 키 없음) → ECR → EKS 무중단 롤링 배포
- 프로덕션급 보안 — 격리 서브넷, IRSA 최소권한, 전 구간 KMS 암호화, WAF + 오리진 검증
- 비용 통제 — HA 토크 변수와 apply/destroy 운용으로 학습 기간 월 [\$N] 이내 유지

3. 설계 배경

본 설계는 팀원 일부가 참여했던 2 차 프로젝트 CARE(EC2 Auto Scaling 기반, 부록 A 참조)에서 검증된 패턴을 출발점으로 삼는다. 3 계층 서브넷 분리, fck-nat 기반 저비용 NAT, CloudFront 오리진 검증 헤더, GitHub OIDC 인증, KMS 공용 키 암호화는 그대로 채택한다.

동시에 2 차 프로젝트에서 확인된 한계 — 로컬 tfstate 로 인한 협업 충돌 위험, NAT 인스턴스 아웃바운드 병목, 인스턴스 룰에 집중된 권한 — 는 각각 S3 원격 백엔드(첫 커밋부터), ECR VPC 엔드포인트, IRSA 권한 분리로 구조적으로 해소한다. 코드는 새 팀에서 전부 새로 작성한다.

4. 목표 아키텍처

리전은 ap-northeast-2(서울), 2 개 가용영역을 사용하며 서브넷은 퍼블릭(엣지) / 프라이빗(EKS) / 격리(DB·캐시) 3 계층으로 분리한다.

구간	구성
트래픽 유입	Route 53 → CloudFront(+WAF, X-Origin-Verify 헤더 부착) → ALB(퍼블릭 서브넷, Terraform 관리)
컴퓨트	EKS 클러스터 + 관리형 노드그룹(t3.medium 2~6 대). ALB 와는 ip 타입 Target Group + TargetGroupBinding 으로 연결
클러스터 애드온	AWS Load Balancer Controller, External Secrets(SSM 연동), EFS CSI, metrics-server, Cluster Autoscaler — 전부 IRSA 기반
데이터 계층	RDS MariaDB(격리 서브넷), ElastiCache Redis(TLS+AUTH), EFS 공유 스토리지
아웃바운드	fck-nat 인스턴스 + ECR·S3 VPC 엔드포인트(이미지 풀 트래픽 우회)

CI/CD	GitHub Actions(OIDC) → 이미지 빌드 → ECR 푸시 → kubectl rollout (실패 시 자동 undo)
관측	CloudWatch Container Insights, [Prometheus/Grafana — 팀 논의]

DR: [] 포함 여부 논의. 클러스터 재생성 절차를 Terraform 모듈 + 런북으로 문서화하는 방식을 제안한다.

5. 핵심 기술 결정 (ADR 요약)

각 결정은 레포의 docs/adr/ 디렉터리에 상세 기록한다.

ADR-001 컨트롤 플레인 — EKS 채택

셀프호스팅(k3s/kubeadm) 대비 etcd 백업·인증서·업그레이드 운영 부담을 제거하고 애플리케이션 계층에 집중한다. 컨트롤 플레인 비용은 apply/destroy 운용으로 통제한다.

ADR-002 ALB 관리 주체 — Terraform ALB + TargetGroupBinding

Ingress 로 ALB 를 생성하면 ALB 생명주기가 클러스터에 종속되고 CloudFront 오리진 도메인이 변할 수 있다. Terraform 이 ALB 와 오리진 검증 물을 소유하고, 클러스터는 TGB 로 Pod 만 등록한다.

ADR-003 노드 오토스케일러 — Cluster Autoscaler

단일 노드그룹 2~6 대 규모에서는 CA 가 예측 가능하고 충분하다. 스팟 혼합·다중 인스턴스 타입이 필요해지는 시점에 Karpenter 를 재검토한다.

ADR-004 시크릿 주입 — External Secrets Operator + SSM Parameter Store

앱과 인프라의 결합을 제거한다. 로테이션이 필요해지면 Secrets Manager 승격을 논의한다. []

ADR-005 노드 타입 — t3.medium × 2 (prefix delegation 활성화)

t3.small 은 시스템 Pod 오버헤드를 제외하면 앱 가용량이 부족하다. prefix delegation 으로 노드당 Pod 상한을 확보한다.

6. 레포 및 협업 구조

모듈	범위	오너
network	VPC, 서브넷, NAT, VPC 엔드포인트	[A]
data	RDS, Redis, EFS, S3	[B]
eks	클러스터, 노드그룹, IRSA, 애드온	[C]
edge	CloudFront, WAF, ALB, Route53	[D]

- state: S3 백엔드 + 락을 첫 커밋부터 적용. 초기에는 단일 state 로 운영(규모상 충분)
- 브랜치: main 보호 + PR 필수. PR 마다 terraform plan 자동 코멘트, tfsec 보안 스캔, Infracost 비용 diff
- 배포: main 머지 시 apply. 애플리케이션은 별도 워크플로로 이미지 빌드 → rollout
- k8s/ 디렉터리에 Deployment, HPA, TargetGroupBinding, ExternalSecret 매니페스트 관리

7. 비용 계획

"상시 가동"이 아닌 "필요할 때만 가동"을 기본 전략으로 한다. 전체 스택 기준 시간당 약 \$0.5~0.6(EKS 컨트롤 플레인 + 노드 2 대 + RDS Single-AZ + 엣지 계층)로, 주 20 시간 작업 시 월 \$50 내외를 예상한다.

- enable_ha 변수 하나로 RDS Multi-AZ / Redis 레플리카 / 삭제 보호를 일괄 제어 — 평소 off, 데모·발표 시 on
- fck-nat + ECR VPC 엔드포인트로 NAT 관련 고정비·전송비 최소화 (NAT GW 대비 월 약 \$42 절감)
- AWS Budgets 예산 알람 설정, 주간 Cost Explorer 리뷰 담당 지정
- 상세 견적: AWS 비용계산기 입력 시트 기반으로 [담당자] 산출 → 팀 예산 [\$N/월] 확정 []

8. 일정 (초안)

주차	마일스톤	내용
1 주	기반	레포·S3 백엔드 셋업, 모듈 뼈대, 네트워크 + 데이터 계층 apply
2 주	EKS	클러스터·노드그룹·애드온 구성, 애플리케이션 컨테이너화
3 주	개통	CI/CD 파이프라인, 엣지 계층 연결, e2e 트래픽 개통
4 주	검증	HPA/CA 부하 테스트, 장애 주입 테스트, 관측 대시보드
5 주	마무리	문서화(ADR·트러블슈팅·다이아그램), 발표 자료, 리허설

9. 검증 시나리오 (발표 데모)

- 스케일링 실측 — k6 부하 주입 → HPA 가 Pod 2→8 증설 → 노드 부족 → CA 가 노드 2→4 증설. Grafana 그래프로 제시
- 무중단 배포 — 배포 진행 중 초당 응답률 그래프로 rollout 전후 에러율 0 확인
- 장애 주입 — 노드 1 대 강제 종료 → Pod 재스케줄링 → 서비스 무영향 확인
- 보안 검증 — ALB DNS 직접 호출 시 403(오리진 검증), IRSA 권한 밖 API 호출 실패 로그

10. 리스크

리스크	대응
K8s 학습 곡선으로 일정 지연	1~2 주차 로컬 kind/minikube 병행 학습, 모듈 담당과 K8s 담당 분리
비용 초과	Budgets 예산 알람 + 주간 Cost Explorer 리뷰 담당 지정
팀원 간 Terraform 충돌	S3 백엔드 + PR plan 필수로 구조적 방지
destroy/apply 반복 시 데이터 유실	RDS 스냅샷 복원 시드 절차 문서화

다음 버전에서 확정할 것: 서비스명·기능 범위, 팀 역할 분배, DR 포함 여부, 관측 스택(Grafana 여부), 예산 상한.

부록 A. 참조 아키텍처 (2 차 프로젝트)

아래는 설계 배경(3 장)에서 언급한 2 차 프로젝트의 아키텍처다. 본 프로젝트의 네트워크·엣지·데이터 계층 패턴의 출처이며, 컴퓨트 계층(EC2 ASG)과 배포(CodeDeploy)는 본 프로젝트에서 EKS 기반으로 대체된다.

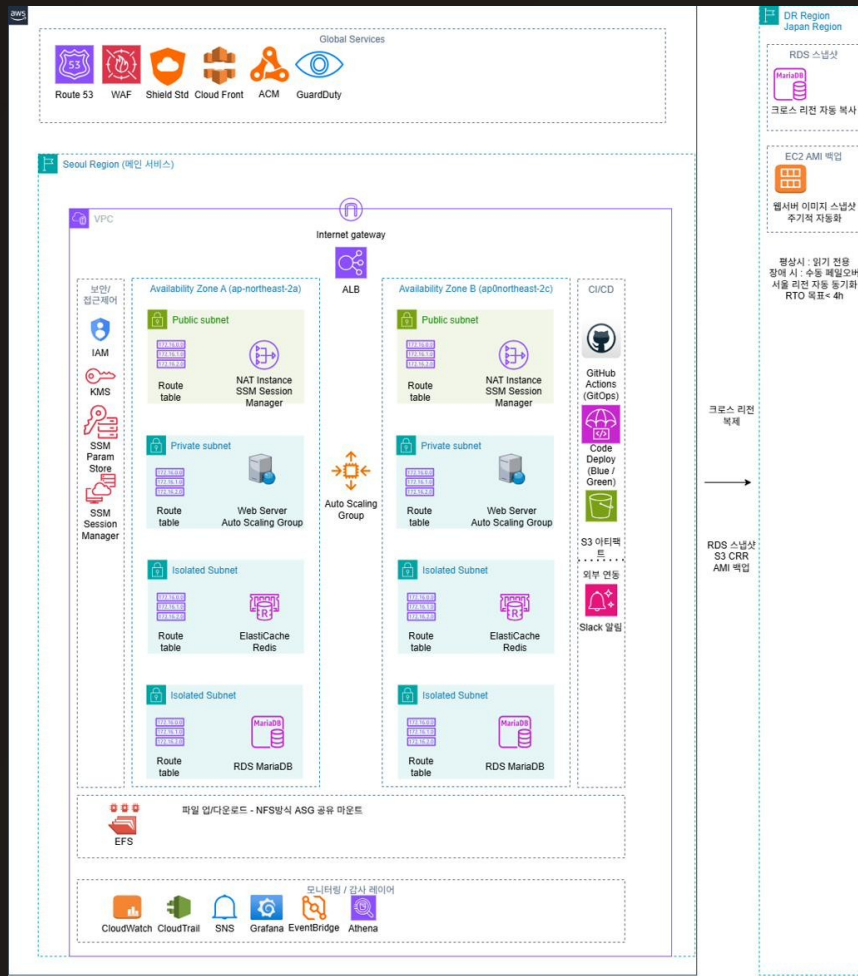


그림 A-1. 2 차 프로젝트(CARE) 아키텍처 — draw.io 원본